OT

OpsTracker

Construction Operations Tracker — Complete System Documentation

React 19 · Express 5 · PostgreSQL

Role-Based Dashboards

Equipment Management

Vercel Deployment

Table of Contents

> *A full-stack web platform for managing construction site operations: projects, workforce, materials, equipment, attendance, documents, and analytics — with role-based dashboards and real-time in-app notifications.*

System Overview

OpsTracker centralizes day-to-day construction operations into a single web application. Field supervisors, managers, and administrators work from role-specific dashboards while workers interact with tasks, attendance, and equipment through simplified views.

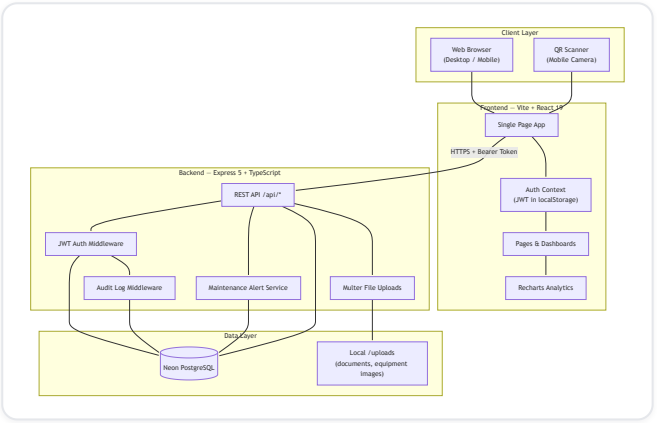

Figure 1: System Architecture Overview

What the system does

Capability	Description
Multi-site operations	Organize work as **Projects → Sites → Teams**
Workforce control	Four roles with scoped permissions and dedicated dashboards
Resource tracking	Materials inventory, requisitions, and per-site stock levels
Equipment lifecycle	Registration, QR codes, usage sessions, maintenance, work orders, spare parts
Workforce attendance	Clock in/out, leave requests, supervisor marking
Documents	Upload permits, plans, and site photos
Notifications	In-app alerts for tasks, stock, equipment, and maintenance
Reporting	Charts for tasks, equipment status, utilization, and attendance
Audit trail	Logs user actions on mutating API requests

Architecture

Request lifecycle

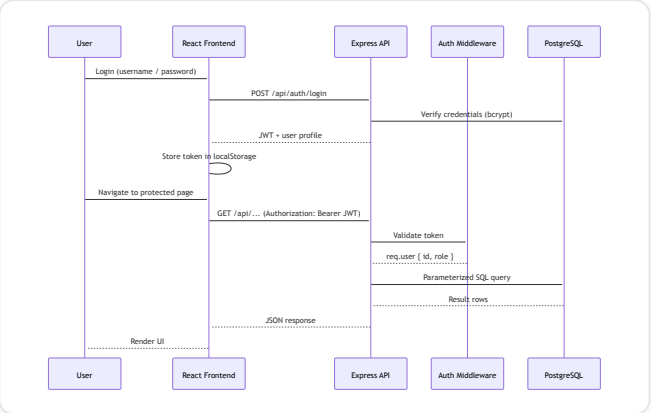

Figure 2: Request Lifecycle (Authentication Flow)

Deployment topology

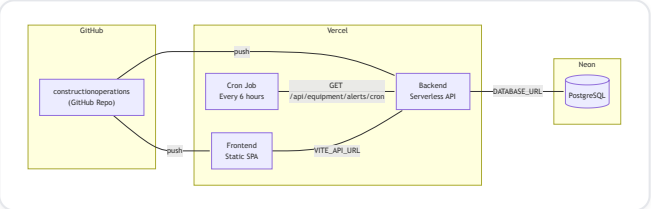

Figure 3: Deployment Topology

User Roles & Access

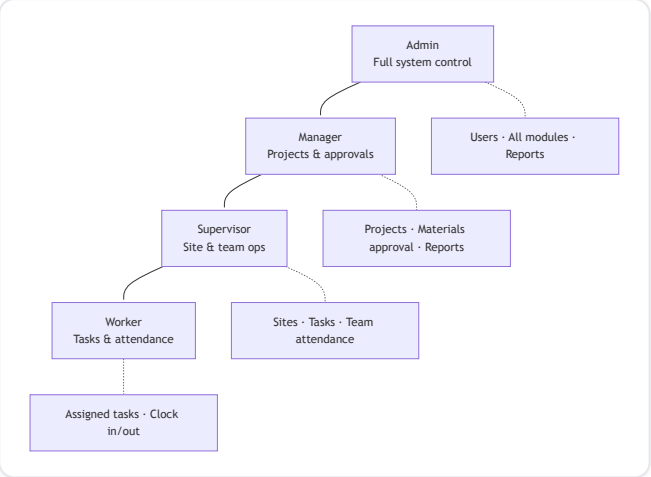

Figure 4: User Roles & Access Hierarchy

Role	Dashboard	Key permissions
Admin	`/dashboard/admin`	User CRUD, projects, sites, reports, equipment, system overview
Manager	`/dashboard/manager`	Project oversight, material requisition approval, reports
Supervisor	`/dashboard/supervisor`	Site teams, task assignment, attendance marking
Worker	`/dashboard/worker`	Own tasks, clock in/out, personal attendance

Admin sidebar is streamlined to: Dashboard, Projects, Sites, Reports, Users.
Operational modules (Tasks, Materials, Equipment, Attendance, Documents) remain available to Manager, Supervisor, and Worker roles.

Feature Modules

Figure 5: Feature Modules Mind Map

Module summary

Module	Frontend route	Backend routes	Highlights
Authentication	`/login`	`/api/auth/*`	JWT, bcrypt passwords
Users	`/users`	`/api/users/*`	Admin-only create/edit/deactivate
Projects	`/projects`	`/api/projects/*`	Status: active, completed, on_hold
Sites	`/sites`	`/api/sites/*`	Worker assignment, supervisor link
Tasks	`/tasks`	`/api/tasks/*`	Priority, progress updates, role filtering
Materials	`/materials`	`/api/materials/*`	Inventory, transactions, requisitions
Equipment	`/equipment`	`/api/equipment/*`, `/api/spare-parts/*`, `/api/work-orders/*`	Full lifecycle (see below)
Attendance	`/attendance`	`/api/attendance/*`	Clock in/out, leave workflow
Documents	`/documents`	`/api/documents/*`	Multer uploads, download
Notifications	`/notifications`	`/api/notifications/*`	Read/unread, type filtering
Reports	`/reports`	`/api/reports/*`	Recharts visualizations

Equipment Management (Extended)

The equipment module is a dedicated sub-system covering registration through retirement.

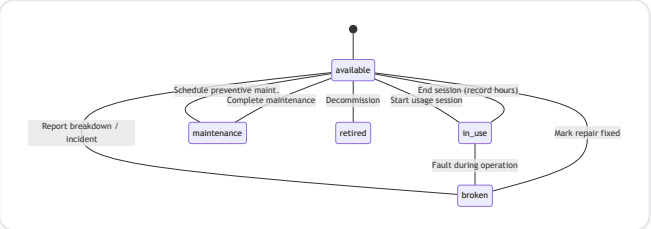

Figure 6: Equipment Status State Machine

Equipment UI tabs

Tab	Purpose
Registry	CRUD, images, documents, QR display, detail panel
Tracking	Assignments, active usage sessions, transfer history
Maintenance	Schedule preventive/corrective, assign technicians, complete with spare parts
Work Orders	Create, assign, track, complete with detailed notes
Spare Parts	Inventory, edit/delete, consumption history, reorder alerts
Incidents	Breakdown, fault, accident, damage reports + repair progress
Utilization	Usage hours chart, idle detection, 30-day performance

QR code workflow

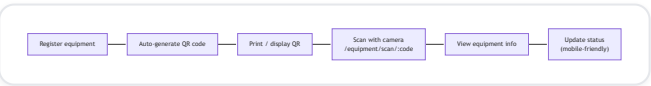

Figure 7: QR Code Workflow

Equipment API highlights

Endpoint	Method	Description
`/api/equipment`	GET, POST	List / register equipment
`/api/equipment/:id`	GET, PUT, DELETE	Detail, update, delete
`/api/equipment/:id/qr`	GET	QR code image (data URL)
`/api/equipment/scan/:code`	GET	Lookup by QR token
`/api/equipment/:id/usage`	POST	Start usage session
`/api/equipment/usage/:id/end`	PATCH	End session + record hours
`/api/equipment/:id/transfer`	POST	Record location transfer
`/api/equipment/transfers/all`	GET	Fleet transfer history
`/api/equipment/:id/maintenance`	POST	Schedule maintenance
`/api/equipment/maintenance/:id`	PATCH	Complete (+ spare parts)
`/api/equipment/:id/breakdown`	POST	Report incident
`/api/equipment/utilization/report`	GET	Utilization analytics
`/api/equipment/alerts/check`	POST	Trigger in-app alerts
`/api/spare-parts/*`	CRUD	Spare parts inventory
`/api/work-orders/*`	CRUD	Maintenance work orders

Tech Stack

Frontend

Technology	Version	Purpose
Vite	7.x	Build tool & dev server
React	19.x	UI framework
TypeScript	5.9	Type safety
React Router	7.x	Client-side routing
Tailwind CSS	3.4	Utility-first styling
Axios	1.x	HTTP client + interceptors
Recharts	3.x	Dashboard & report charts
Lucide React	0.56x	Icon system
html5-qrcode	2.x	Camera QR scanning
date-fns	4.x	Date formatting

Backend

Technology	Version	Purpose
Node.js	18+	Runtime
Express	5.x	HTTP server & routing
TypeScript	5.9	Type safety
pg	8.x	PostgreSQL client
jsonwebtoken	9.x	JWT authentication
bcryptjs	3.x	Password hashing
express-validator	7.x	Input validation
Multer	2.x	Multipart file uploads
qrcode	1.x	QR code generation

Infrastructure

Service	Role
Neon PostgreSQL	Serverless PostgreSQL database
Vercel	Frontend & backend hosting
GitHub	Source control & CI trigger

Project Structure

```
constructionoperations/
├── backend/
│   ├── api/
│   │   └── index.ts              # Vercel serverless entry point
│   ├── src/
│   │   ├── config/
│   │   │   ├── database.ts       # pg connection pool
│   │   │   ├── initDatabase.ts   # Base schema bootstrap
│   │   │   └── equipmentSchema.ts # Equipment module migrations
│   │   ├── middleware/
│   │   │   ├── auth.ts           # JWT authenticate + authorize
│   │   │   └── audit.ts          # Mutation audit logging
│   │   ├── routes/               # REST route modules (12+)
│   │   ├── services/
│   │   │   ├── maintenanceAlerts.ts
│   │   │   └── notificationService.ts
│   │   ├── scripts/              # Seed & admin creation
│   │   ├── utils/                # JWT, password helpers
│   │   └── index.ts              # Local dev server entry
│   ├── uploads/                  # Uploaded files (documents, equipment)
│   ├── package.json
│   └── vercel.json
│
├── frontend/
│   ├── public/                   # logo.svg, favicon
│   ├── src/
│   │   ├── components/           # Layout, ProtectedRoute
│   │   ├── contexts/             # AuthContext
│   │   ├── lib/                  # api.ts, auth.ts, equipmentTypes.ts
│   │   ├── pages/
│   │   │   ├── dashboards/       # Admin, Manager, Supervisor, Worker
│   │   │   ├── Equipment.tsx     # Full equipment module (tabs)
│   │   │   ├── EquipmentScan.tsx # QR camera scanner
│   │   │   └── ...               # Projects, Sites, Tasks, etc.
│   │   ├── App.tsx               # Route definitions
│   │   └── main.tsx
│   └── package.json
│
├── README.md
├── FEATURES.md
├── SETUP.md
├── DASHBOARDS.md
└── VERCEL_DEPLOYMENT.md
```

Database Schema

Schema is **auto-created on startup** via `initializeDatabase()` + `migrateEquipmentSchema()`. No separate migration CLI is required — run the backend or execute the init script.

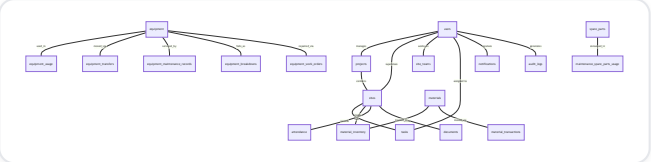

Figure 8: Database Entity Relationships

Core tables (25+)

Group	Tables
Identity	`users`
Operations	`projects`, `sites`, `site_teams`, `tasks`, `task_updates`, `daily_activities`
Materials	`materials`, `material_inventory`, `material_transactions`, `material_requisitions`
Equipment	`equipment`, `equipment_usage`, `equipment_transfers`, `equipment_documents`, `equipment_maintenance_records`, `equipment_breakdowns`, `equipment_work_orders`, `spare_parts`, `maintenance_spare_parts_usage`
Workforce	`attendance`, `leave_requests`
System	`documents`, `notifications`, `audit_logs`, `notification_delivery_log`

Run migrations manually

```
cd backend
npm install
DATABASE_URL="your-neon-connection-string" npx ts-node -e "
  import { initializeDatabase } from './src/config/initDatabase';
  initializeDatabase().then(() => process.exit(0)).catch(e => { console.error(e); process.ex
"
```

Or seed demo data:

```
cd backend
npm run seed
# Default users: admin/admin123, manager/manager123, supervisor/supervisor123, worker/worker
```

API Reference

Base URL: `http://localhost:5000/api` (local) or your Vercel backend URL.

All protected routes require header: `Authorization: Bearer <JWT_TOKEN>`

Authentication

Method	Endpoint	Description
POST	`/auth/register`	Register new user
POST	`/auth/login`	Login → returns JWT
GET	`/auth/me`	Current user profile

Users

Method	Endpoint	Access
GET	`/users`	Admin, Manager
POST	`/users`	Admin
PUT	`/users/:id`	Admin
PATCH	`/users/:id/deactivate`	Admin

Projects & Sites

Method	Endpoint	Description
GET/POST	`/projects`	List / create projects
PUT	`/projects/:id`	Update project
GET	`/projects/:id/sites`	Sites under project
GET/POST	`/sites`	List / create sites
PUT	`/sites/:id`	Update site
POST	`/sites/:id/assign-worker`	Add worker to site team

Tasks

Method	Endpoint	Description
GET	`/tasks`	List (role-filtered)
POST	`/tasks`	Create task
PUT	`/tasks/:id`	Update task
POST	`/tasks/:id/updates`	Log progress update

Materials

Method	Endpoint	Description
GET	`/materials`	List materials
GET	`/materials/inventory/:site_id`	Site inventory
POST	`/materials/transactions`	Record delivery/usage
POST	`/materials/requisitions`	Request materials
PATCH	`/materials/requisitions/:id`	Approve / reject

Attendance

Method	Endpoint	Description
GET	`/attendance`	List records
POST	`/attendance/clock-in`	Clock in
POST	`/attendance/clock-out`	Clock out
POST	`/attendance/leave-requests`	Submit leave

Documents

Method	Endpoint	Description
GET	`/documents`	List documents
POST	`/documents/upload`	Upload file (multipart)
GET	`/documents/:id/download`	Download file

Notifications & Reports

Method	Endpoint	Description
GET	`/notifications`	User notifications
PATCH	`/notifications/:id/read`	Mark read
GET	`/reports/dashboard`	Role-based dashboard stats
GET	`/reports/tasks/progress`	Task status breakdown
GET	`/reports/equipment/status`	Equipment status counts
GET	`/reports/equipment/utilization`	Usage hours analytics
GET	`/reports/attendance/summary`	Attendance report

> *Full equipment, spare parts, and work order endpoints are listed in Equipment Management.*

Getting Started

Prerequisites

- **Node.js** 18 or higher
- **npm** 9+
- **Neon PostgreSQL** database (or any PostgreSQL 14+ instance)

1. Clone the repository

```
git clone https://github.com/bvggies/constructionoperations.git
cd constructionoperations
```

2. Backend setup

```
cd backend
npm install
```

Create `backend/.env`:

```
DATABASE_URL=postgresql://USER:PASSWORD@HOST/DATABASE?sslmode=require
JWT_SECRET=change-this-to-a-long-random-secret
PORT=5000
NODE_ENV=development
FRONTEND_URL=http://localhost:5173
```

Start the API (runs migrations automatically):

```
npm run dev
# → http://localhost:5000
```

3. Frontend setup

```
cd frontend
npm install
```

Create `frontend/.env`:

```
VITE_API_URL=http://localhost:5000/api
```

Start the dev server:

```
npm run dev
# → http://localhost:5173
```

4. Seed demo data (optional)

```
cd backend
npm run seed
```

User	Password	Role
admin	admin123	Admin
manager	manager123	Manager
supervisor	supervisor123	Supervisor
worker	worker123	Worker

Environment Variables

Backend

Variable	Required	Description
`DATABASE_URL`	Yes	Neon PostgreSQL connection string
`JWT_SECRET`	Yes	Secret for signing JWT tokens
`PORT`	No	Server port (default: `5000`)
`NODE_ENV`	No	`development` or `production`
`FRONTEND_URL`	No	Used in QR code scan URLs
`CRON_SECRET`	Prod	Secret for Vercel cron alert endpoint
`SEED_SECRET`	No	Protects `POST /api/seed` endpoint

Frontend

Variable	Required	Description
`VITE_API_URL`	Yes	Backend API base URL (must include `/api`)

Deployment

Both frontend and backend deploy independently to **Vercel**. See VERCEL_DEPLOYMENT.md and VERCEL_ENV_SETUP.md for detailed guides.

```
# Backend
cd backend && vercel

# Frontend
cd frontend && vercel
```

Vercel cron (maintenance alerts)

The backend `vercel.json` includes a cron job that hits `/api/equipment/alerts/cron` every 6 hours. Set `CRON_SECRET` in your Vercel project environment — Vercel sends it as a Bearer token automatically.

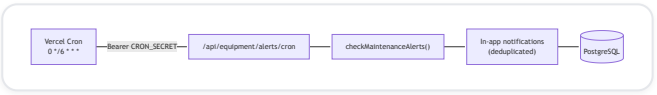

Figure 9: Vercel Cron — Maintenance Alerts

Security

Measure	Implementation
Authentication	JWT bearer tokens
Password storage	bcrypt hashing
Authorization	Role-based middleware on routes
SQL injection	Parameterized queries (`pg`)
Input validation	express-validator on POST/PUT bodies
CORS	Enabled for frontend origin
File uploads	Type & size limits via Multer
Audit trail	All non-GET mutations logged
Secrets	Never commit `.env` files — use Vercel env vars

License

MIT © bvggies

OpsTracker — Built for construction teams who need clarity across every site, shift, and asset.